Claude Science — Detailed Usage Guide (CSRTB v2.11)

> **This is the human-readable twin of an authoritative machine root.** The source of record is `../machine_md/CS_USAGE_DETAILED.machine.md`; this `.md` and any rendered PDF are derived from it by atom-preserving translation. If the two ever disagree, the machine root wins — corrections land there first, then propagate here. This is a bundle-dir reference document: it is read by a person or a session, is not installed into your Claude Science account, and touches none of the bundle's build machinery.

Who this is for: a scientist who can open a Claude Science conversation but knows little else. It teaches Claude Science from near-zero, then how the Research Toolkit **bundle** augments it, then how to use the whole thing for research.

How it is organized: the skeleton *is* Claude Science's functional architecture — its primitives. Learn the primitives and you can reason about what the system can do instead of memorizing a feature list. The guide runs **MAP** (the mental model) → **Part A** (base Claude Science, function by function) → **Part B** (the bundle overlay) → **Part C** (in practice).

The document set — cross-reference only these: **CS_README** (the front door), **CS_QUICKSTART** (day-one, read it first if you are new), **CS_USAGE_DETAILED** (this reference), **CS_ADVANCED** (the deep dive), and **CS_INSTALL_STARTER_v2.11** (the paste-ready install, kept at the bundle root).

MAP · What Claude Science is, and its architecture

Two guides ship together. **CS_QUICKSTART** is the day-one page (one to two pages — read it first if you are new); this **detailed** guide is the reference. Between them they teach Claude Science fundamentals, the bundle's augmentations, and research use. Both are machine→human→PDF artifacts, so what you are reading is the human render of a `.machine.md` root. When a term is unfamiliar, jump to the glossary at the end.

What it is. Claude Science is an agentic research *environment*: a conversation joined to a Python repl — a persistent kernel — that runs code, reads and writes durable **artifacts**, and dispatches specialist agents, all inside a remote sandbox. It is not a chat box. It runs real cells and produces real artifacts, and every step is visible in the transcript.

How it runs — the turn loop. You state a goal; it loads context (including automatically recalled project memory); it plans; it runs repl cells and calls the `host.*` API within your account's permissions; you review; and the cycle repeats. Every capability below is one layer this loop passes through.

The primitives are the whole mental model, and Part A takes them one at a time:

- **Context** — what Claude *sees*: the live window, automatically recalled project or profile **memory**, the skills and kernels loaded this session, and search.

- **Instructions** — how you *steer* it: the **profile** it wears, the model and tier routing, the skills that auto-load, and the agency dial.
- **Actions and guardrails** — what it *does* and the boundary it sits inside: the `host.*` API and the repl, your account permissions, per-host network grants, and sandbox isolation.
- **Delegation** — handing bounded jobs to specialist **profiles** through `host.delegate`, and running work in the background or in parallel.
- **The durable record** — **artifacts** and **lineage**, the persistent and reproducible store; this is the Claude Science analog of files plus version control.
- **Automation** — **kernel** sidecars: loaded gate and helper functions that you (or a skill) *call*. This is the Claude Science analog of Claude Code's event-hooks, but it fires at call time rather than being run by the harness.

One note on what is *absent*: base Claude Science has no `~/.claude`-style scope-merge of settings files. The toolkit installs into your **account** (skills and profiles), and durable steering lives in **memory** rather than in a per-repository `CLAUDE.md`.

The invariant to carry: everything Claude Science can do is one of these primitives inside the turn loop. Learn the primitives, derive the features.

PART A · Base Claude Science, by function

Each unit below covers one primitive in a fixed arc: what it is **for**, a **handle** to hold it by, its mechanics, the one-line **invariant**, and what it **couples** to.

A · The loop, and driving it

This is the driver's seat — running and steering Claude turn by turn. The handle: pair-science out loud, where you set direction, it acts, you correct, and the cycle repeats.

You start by opening a Claude Science conversation inside a **project**. The project scopes what persists: its artifacts (with their lineage) and its memory carry across every session in it. Work then runs as **cells** in a persistent Python kernel in a remote sandbox — Claude writes a cell, runs it, reads the output, and iterates. You see each cell and its result in the transcript; nothing is hidden. Kernel state (variables, imports) persists across cells within a session until the kernel restarts.

At the prompt, be specific and name your deliverables — for example, "fit a `bam` AR1 model to artifact `x_v3` and save the diagnostic plot as an artifact." Reference artifacts by id or name rather than pasting their contents. You review each turn against the transcript, which scrolls the full action history with every cell and output shown.

You can steer mid-task by typing a correction while a step runs; a running background job or a delegated child picks up the steer at its next tool round. If it is going wrong, restate the goal — that beats letting it run. Claude is interactive by default and pauses at genuine decision points. The agency-dial skills set that posture: `/collab` is the default and surfaces the non-trivial calls; `/solo` runs to completion with no check-ins; `/plan` maps the work and gets your go/no-go before any scope-defining act. In short: the default is brisk but interactive, `/solo` runs to completion, and `/plan` deliberates first.

Invariant: it works in reviewable repl turns that produce real artifacts; you can redirect at any point, and nothing is hidden. This couples to the agency dial (Instructions and Part B), to the artifacts a cell writes (the durable record), and to the memory recalled on each turn (Context).

B · Context — what Claude sees

Context is Claude's working memory for the turn. The handle: a desk with a permanent shelf that auto-restocks (memory) and a workbench that fills up and gets tidied (the window).

There are two kinds. The **persistent** kind is project or profile **memory**, and it survives across sessions: memory automatically recalls into context each turn, which is how durable preferences, rules, and lessons reach Claude — the Claude Science analog of Claude Code's always-loaded `CLAUDE.md`. Each profile accrues its own memory, and project memory is shared across the project's conversations. The **recomputed** kind is the live window: this session's turns, cell outputs, artifacts read, and search results. It is large but finite.

One discipline matters more here than anywhere else. On Claude Science, **memory is the poison surface**: only memory auto-recalls, whereas artifacts stay inert until searched or opened. A stale memory row re-injects itself as "current" every turn. So keep one canonical memory row per topic — supersede it in place, and retract the old claim when it changes — and keep done-records and closed state as inert **artifacts** rather than memory rows. (The `durable-doc-architecture` and `provenance-over-description` skills in Part B own this discipline.)

Retrieval is by **search**: an artifact contributes to context only when it is found, so name and tag artifacts such that the search that ought to pull them actually does. As the window fills, Claude auto-summarizes older turns — you keep the thread, but detail can blur. For long or complex effort, write a cold-start brief with `/handoff-brief` before the window fills; the brief is a pointer that lets the next session load targeted, referencing canonical artifacts by id (see Delegation).

Invariant: persistent context is memory (it auto-recalls, and it is the poison surface), while the live window is summarized as it fills — so protect long work with `/handoff-brief`, keep one canonical memory row per topic, and let closed state live in inert artifacts. This couples to Part B's currency skills, to `/handoff-brief` under Delegation, and to the artifacts-versus-memory distinction in the durable record.

C · Instructions — how you steer it

Instructions are the dials that direct *how* Claude works: the profile it wears, the model and reasoning budget, and the skills it fires. The handle: a control panel where you pick the specialist persona (the profile), the engine (the model), the gear (the effort), and the tool you reach for (the skill).

Profiles are the Claude Science analog of a Claude Code subagent's specialized prompt — but here they can shape the main persona too. A profile is a system-prompt persona with a curated skill set and tool access. You can pick a profile to specialize the whole conversation, or dispatch one as a delegated child (see Delegation). The bundle ships eighteen of them (Part B); `GENERALIST` is the wide-reach daily driver, and the rest are domain specialists.

For **models**, the capability ladder runs from hardest to cheapest: Fable 5 for the hardest reasoning and modeling, Opus 4.8 as the capable workhorse, Sonnet for fast routine coding, and Haiku as the fastest and cheapest for trivial work. The model policy is binding: **never** Claude Opus 5 (`claude-opus-5`), and never the bare `opus` alias, which resolves to Opus 5 — always use full ids such as `claude-opus-4-8` and `claude-fable-5`. The ban applies to delegated children and to probes. When you delegate, per-child routing uses a difficulty **tier → model** table (the `delegation-planning` kernel's `TIER_TABLE` and `resolve_tier`), and `host.delegate(model=…)` sets a child's model.

Effort is the reasoning spent before acting: higher is better on hard problems and slightly slower. Stay high for research, modeling, and debugging, and lower it only for bulk mechanical work.

Skills are auto-loading capabilities, and they are the one steering mechanism on Claude Science — there is no separate slash-command CLI layer. Describe a task and the matching skill auto-loads through `host.skills` on its trigger; a few skills (the agency dial and some workflow skills) you also fire by name, such as "switch to `/plan`." A skill may ship a **kernel** sidecar (see Automation). The fifty-two bundle skills are cataloged by family in Part B — route by *description*, not by guessing at names.

You can also steer persistently: write a durable preference or rule into memory and it auto-recalls every turn (Context). The bundle's discipline skills ride that same auto-load mechanism (Part B).

Invariant: pick a profile for the persona, a model by difficulty (never Opus 5), and let skills auto-load on their trigger (or name one) — the profile, skills, and model together are the steering. This couples to the tier table and Opus-5 ban (Part B and Delegation), to the fifty-two skills and eighteen profiles as an overlay (Part B), and to delegating a profile (Delegation).

D · Actions and guardrails

This unit covers what Claude *does* — run cells, call the `host.*` API, read and write artifacts — and the boundary every action sits inside. The handle: a workshop sealed in a clean room, where the tools are powerful but the room's walls (the sandbox) and its supply lines (network grants) define what can reach in or out.

The **`host.*` API** is the action surface. `host.delegate` dispatches profiles (Delegation). `host.skills` reads, loads, and edits skills and kernels. `host.agents` is the set of installed profiles. `host.artifacts` is the durable store (the durable record). `host.lineage` returns the reproduction code for an artifact version (the durable record). `host.frames` and `host.query` expose the session record — events and prior turns. `host.llm` makes a raw model call. `host.get_local_compute_stats` reports compute headroom (Delegation).

The **guardrails** are Claude-Science-true, and they are neither a Claude Code deny-list nor hooks. First, **sandbox isolation**: cells run in a remote sandbox with no access to your local machine's files or audio device by default — which is why an "alert when done" has to reach you through the browser (the `audible-alert` skill) rather than a local beep. Second, **network grants**: outbound network is gated per host, so a skill that needs the network — for instance, `folio-science`'s Kroki diagram render reaching `kroki.io` — requires an explicit grant, and without it the call is blocked.

Third, **account permissions**: some operations run against your live account or local machine rather than the sandbox — for example, `preflight-parallel` routes filesystem-heavy inspection to the `mac-local` path instead of the sandbox — so they run outside the sandbox by design and are gated accordingly.

The **plan posture** is set by `/plan` (a skill, Part B), the deliberation-max detent: for a scope-defining or expensive step it maps the territory and gets your go/no-go before committing, while cheap, local, reversible acts still proceed. It is the Claude Science analog of Claude Code plan mode — a skill, not a harness mode.

Invariant: Claude acts through the `host.*` API and the repl, inside a sandbox whose walls are isolation, per-host network grants, and account permissions. It acts freely inside the boundary and never around it, and there is no deny-list to override because the boundary is structural. This couples to `host.delegate` (Delegation), to `host.artifacts` and `host.lineage` (the durable record), and to the `/plan` posture (Instructions and Part B).

E · Delegation and scale

Delegation keeps the main thread clean by handing bounded jobs to specialist **profiles**, and it lets you get more done at once through background and parallel runs. The handle: running a lab, where you (the lead) delegate specialized tasks to specialists and start long instruments running while you keep working.

You delegate with `host.delegate`, dispatching one or more child agents — each a profile with its own fresh context, a bounded task, and a model. `host.delegate([{task, name, model}, …])` runs children, parallel by default; you then collect their results and read the artifacts they wrote. Children are steerable mid-run (a message lands at the child's next tool round), and a child returns its report once — so a redesign means re-dispatching, while a correction is a queued steer.

There are four **topologies**, owned by `delegation-planning`: a parallel-wave (independent fan-out), a sequential-build (each stage feeds the next), a convergence (many drafts collapsing into one synthesis), and a verify-loop (a builder paired with an adversarial reviewer). Rule a cascade out for tightly-coupled single-thread work.

For **background and asynchronous** work, long compute — model fits, bootstraps, simulations — runs detached while Claude keeps working; a running job never blocks, because Claude advances a different thread meanwhile. Confirm "done" from the job's own artifacts or record (`host.frames`, the written artifact), not from elapsed time — silence does not mean done (this is the `verification-loop` discipline, Part B). For **parallelism**, `preflight-parallel` sizes concurrency correctly: it measures per-unit cost, reads real headroom from `host.get_local_compute_stats()` (instantaneous cores and RAM, not load average), dispatches detached with briefs persisted first, and then batch-analyzes.

Invariant: delegate bounded work to a specialist profile (a clean main thread) and run independent jobs in parallel in the background — but read "done" from the job's own artifacts, never the clock, and never route a child to Opus 5. This couples to profiles and tier→model

routing (Instructions), to the artifacts children write and you collect (the durable record), and to `preflight-parallel`, `supervisory-workflow`, and `directing-execution` (Part B).

F · The durable record — artifacts and lineage

This is the persistent, reproducible store: how work survives a session and stays re-runnable, the Claude Science analog of files plus version control. The handle: a lab notebook with a photocopier, where every result is filed alongside the exact procedure that made it.

Artifacts are durable objects — data, figures, docs, fit objects — that Claude writes and retrieves through `host.artifacts`, and they persist across sessions in the project. Each carries **version ids**, and `latest` is last-writer-wins, so pin an explicit `version_id` when a downstream step must read a specific version (this is the `durable-doc-architecture` discipline). **Lineage** goes further: `host.lineage[version_id]["code"]` returns the reproduction code that produced an artifact version — the primary record of what the system actually did. To answer "what does it do now" or "which method is canonical," read the lineage rather than a docstring, memory row, or handoff that only *describes* it (this is `provenance-over-description`).

Provenance hygiene closes the loop: save every intermediate that another cell will read as an artifact *before* you fit, because a file written to `/tmp` is lost on kernel restart and never enters lineage. The `provenance-guard` skill guards this with a `/tmp` linter and a `checkpoint_frame` helper; hand off between kernels through `./handoff/`, never `/tmp`.

Invariant: results live as artifacts with lineage, not loose files; pin explicit version ids for anything downstream; and answer "what is it now" from the lineage (the record), never from a description of it. This couples to the fact that artifacts are inert until searched (Context), to children's outputs (Delegation), and to the provenance and currency skills of Part B (`provenance-guard`, `provenance-over-description`, `durable-doc-architecture`).

G · Automation — kernel sidecars

Automation on Claude Science is the set of reusable, deterministic gate and helper functions a skill ships and Claude *calls* — the analog of Claude Code event-hooks, but at call time rather than harness-fired. The handle: a bench of calibrated jigs that you pick up and use at the right step; a jig does not fire itself.

Kernels are the mechanism: nineteen of the fifty-two skills ship a `kernel.py` sidecar that auto-loads with the skill and exposes named functions. You load one explicitly with `exec(host.skills.read("<skill>", "kernel.py")["content"])` and then call the function. Examples, all from the bundle: `verify_claims()` and `require_receipt()` (verification-loop); `model_route_gate()` and `resolve_tier()` (delegation-planning); `confirm_before_stop()` (directing-execution); `verify_before_assert()` and `checkpoint_frame()` (provenance-guard); `emit_alert()` (audible-alert); and `render_doc()` and `qa_pdf()` (folio-science).

Here is what replaces Claude Code hooks. Claude Code fires hooks on *events* — the harness runs them. Claude Science has no event-hook layer; instead, the kernel gates fire when *invoked* — you call them in a cell, or a skill's own procedure calls them before it emits a claim. So "automation"

on Claude Science is a called gate, not an ambient trigger. A gate returns a verdict plus a marker (such as `[[vloop:…]]`) so an auditor can tell "checked and clean" apart from "never ran."

Invariant: automation on Claude Science is kernel functions you *call* at the right step, not events the harness fires — so the discipline is to actually invoke the gate before the claim ships; a gate that never ran protects nothing. This couples to the verification kernels that encode the always-on discipline (Part B and Part C) and to sidecar authoring rules (`toolkit-extension-authoring`, Part B, plus the sidecar contract below).

PART B · The bundle overlay (deltas on base Claude Science)

Base Claude Science works without any of this. The Research Toolkit **bundle** (CSRTB v2.11) is an *overlay* that specializes it for tower and flux research and for verification-disciplined multi-agent work. Nothing here changes the primitives; it pre-loads specialist profiles, domain skills, and kernel gates onto them.

Counts, recomputed from `crt_science_bundle.json` on 2026-08-02: **52 skills**, **18 profiles**, and **19 of the skills ship a kernel sidecar**. Do not carry the Claude Code toolkit's own roster figures across — that is a different carrier.

The full roster is authoritative only from the primary record — do not hand-copy a table that drifts. The record is the live `host.skills` and `host.agents` enumeration in your session, together with the shipped `crt_science_bundle.json`. What follows is the **family map** (route by description); the record just named is the exact current membership.

The 18 profiles, by family

Dispatch a profile through `host.delegate`, or wear it as the persona; route by description.

- **Verification and review (2):** `CODE_REVIEW_DEBUGGER` — R, Python, Julia, and MATLAB review, and the adversarial reviewer in a verify-loop; `FORMAL_ARGUMENT_CHECKER` — computes an argument's formal and quantitative claims (deontic validity, base-rate/PPV, signal-detection-theory usage).
- **Writing and docs (4):** `SCIENCE_WRITING_STYLIST` — Schimel OCAR revision; `LLM_DOC_ARCHITECT` — machine-facing docs, agent and skill design, and Claude Code → Claude Science porting; `PROMPT_ENGINEER` — tighten a draft prompt; `DESIGN_RATIONALE_ANALYST` — recover implicit rationale and schema.
- **Planner and general (2):** `PLANNER` — decompose, route, tier, and choose topology (the `/plan` persona); `GENERALIST` — the wide-reach daily driver.
- **Domain modelers and stats (7):** `DYNAMICAL_SYSTEMS_MODELER` — how state *evolves* (ODE/PDE/biogeochem); `ECOPHYSIOLOGY_MODELER` — what the system *should* be (optimality, Farquhar); `ECOSYSTEM_MODEL_TRACER` — CliMA/Emerald solve → recorder → plot tracing; `MACHINE_LEARNING_SCIENTIST` — fusion, large-gap reconstruction, and calibrated uncertainty; `MICROMET_RECONSTRUCTOR` — gappy multi-height micromet to seam-free height-resolved drivers; `ML_HYBRID_PROCESS_MODELER` — physics-informed and gray-box above-to-interior flux mapping; `RESEARCH_STATS_ADVISOR` — the why and which of a method, not the code.

- **Software and build (2):** SOFTWARE_DEVELOPER — build to a spec, then hand to CODE_REVIEW_DEBUGGER; AGENT_TOOLING_ENGINEER — the customization layer itself (skills, profiles, kernels, installers, gates).
- **Data management (1):** RESEARCH_DATA_MANAGER — provenance and four-lifecycle keep-versus-sweep on top of lineage.

The 52 skills, by family

Each skill auto-loads on its trigger; route by description.

- **Verification and integrity loops (8)** — the Claude-Science-distinctive engine: `verification-loop` (verify_claims, require_receipt); `provenance-guard` (the `/tmp` linter, verify_before_assert); `provenance-over-description` (read the record, not the description); `count-enumeration-contagion` (recount before you relay an N); `countermeasure-audit` (measure whether a fix worked); `testing-discipline` (red-before-green, fixtures); `durable-doc-architecture` (one owner per topic, pin version ids); `refusal-recovery` (the refusal ladder).
- **Writing and document engine (9):** `writing-science` (Schimel plus a detector kernel); `machine-md` (LLM-facing doc form); `expert-prose-style`; `teaching-narrative`; `design-rationale`; `doc-pipeline` (machine → human → PDF, gated); `folio-science` (render PDF, docx, diagrams); `eliciting-llm-behavior` (a prompt-technique catalog); `figure-qa` (render-and-look).
- **Planner, supervisory, and orchestration (9):** `delegation-planning` (route, tier, topology); `directing-execution` (the run-time supervise loop); `supervisory-workflow` (the operating logic); `request-archetypes` (task to carriers); `handoff-brief` (the cold-start brief); `preflight-parallel` (concurrency sizing); and the agency dial — `solo`, `plan`, `collab`.
- **Domain modelers and scientific method (17):** for statistical fitting — `brms-hierarchical-fitting`, `mgcv-temporal-gam`, `temporal-block-cv`, `temporal-qc-outlier-detection`, `tz-safe-timestamps`, `gap-fill-imputation`, `aggregation-jensen-bias`, `tree-ensembles`; for scientific ML — `scientific-ml-fundamentals`, `calibrated-uq-for-ml`, `ml-emulator-surrogate`, `multi-source-fusion-bias-correction`, `physics-informed-ml`, `micromet-height-interpolation`; for domain physics — `biosphere-atmosphere-flux-exchange`; for method reproduction — `reproduce-model-from-literature`; for compute — `julia-performance-correctness`.
- **Literature and data curation (3):** `sci-file-index` (a metadata catalog); `sci-library-curate` (dedup and topic-organize); `scanned-pdf-ocr`.
- **Toolkit-builder and extension / install and gates (4):** `toolkit-extension-authoring`; `bash-hook-contract` (Claude Code hook authoring); `software-craft`; `refactoring`.
- **Project-canonical (1):** `km67-canonical-methods` — the km67/Tapajós canonical-method registry with a lineage self-check.
- **Utility (1):** `audible-alert` — the browser-channel "beep when done."

The sidecar contract, in one paragraph

A skill's `kernel.py` sidecar auto-loads with the skill and exposes plain-name gate and helper functions. Its top level is restricted so it publishes cleanly: plain-name `def`s, imports, and *literal*-constant assignments only — no computed values (`re.compile`, `frozenset`, any call), no _-

prefixed names, and no top-level `if` (including the `__main__` guard); the `SKILL.md` `description:` field is at most 1024 folded characters. `check_sidecar_contract.py` must exit 0 before any build. Author a new kernel to this contract from the start (owned by `toolkit-extension-authoring` and `AGENT_TOOLING_ENGINEER`).

Invariant: the overlay is specialist profiles, domain skills, and verification kernels layered onto base Claude Science — remove it and Claude Science still runs; keep it and Claude Science is research-ready and verification-disciplined.

PART C · In practice

Research working patterns (task → bundle response)

- Fitting a hierarchical Bayesian model: describe it and `brms-hierarchical-fitting` fires; for a long fit, go to the background with `preflight-parallel`.
- Fitting a big temporal GAM: `mgcv-temporal-gam` (k-selection, `bam` AR1).
- Gap-filling a driver series: `gap-fill-imputation` (chunk-predict-splice, provenance tiers); for a large multi-year gap or a fusion problem, `MACHINE_LEARNING_SCIENTIST` with `multi-source-fusion-bias-correction`.
- QC on a met or flux series: `temporal-qc-outlier-detection`. Cross-validating autocorrelated data: `temporal-block-cv` (never iid). Joining UTC and local data: `tz-safe-timestamps`.
- Debugging an R or Julia result that looks wrong: `CODE_REVIEW_DEBUGGER` (reproduce-before-fixing, root-before-bandaid).
- Choosing or defending a method: `RESEARCH_STATS_ADVISOR`, in `/plan`.
- Stating "the canonical method for X": `provenance-over-description` (read `host.lineage`) — and for km67, `km67-canonical-methods` first.
- Decomposing a big job across agents: `PLANNER` with `delegation-planning` (route, tier, topology), then `directing-execution` while it runs.
- Handing off or pausing: `/handoff-brief`. Running a handed-off task unattended: `/solo`. Making a shareable PDF or docx of a doc: `doc-pipeline` or `folio-science`.
- Always: save intermediates as artifacts before you fit (never `/tmp`), and run independent runs in parallel and batch the analysis.

A worked session shape

1. Open the project conversation and **pick a profile** — for example, `MICROMET_RECONSTRUCTOR` for a driver-reconstruction task, or `GENERALIST`.
2. State the task concretely: the source artifact id, the method, the output artifact name, and "save plots as artifacts."
3. **Skills fire** on their triggers (say, `temporal-qc-outlier-detection`, `gap-fill-imputation`, `preflight-parallel`); name-fire the agency dial as needed (`/plan` first for anything scope-defining).
4. **Delegate** bounded sub-jobs through `host.delegate` to specialist profiles (a parallel-wave or a verify-loop); a long fit goes to background compute.

5. **Collect** via artifacts and `host.frames` — read "done" from the written artifact, not the clock — and pin the version id that downstream steps depend on.
6. **Record** durable outcomes: one canonical memory row per topic (superseded in place), and closed or done state as an inert artifact rather than a memory row.
7. Before shipping any state claim ("N skills," "the gate passes," "byte-identical"), run the verification kernel (`verify_claims`, `require_receipt`) — see below.

The verification-integrity discipline (the bundle's spine)

Receipts. A "verified," "passed," or "byte-identical" claim carries its receipt — the count, the hash, the exit code, the artifact id — in the same breath. `require_receipt()` (verification-loop) is the return-based gate for a receiptless claim.

No "works" without measurement. A shipped fix defaults to `attempted-untested` efficacy until a measurement says otherwise; existence is not efficacy. `countermeasure-audit` measures failure-class rates before any row is upgraded to `verified-working`, and `require_verification_status()` gates a "this fix works" claim that carries no honest status.

Verify before assert. Every asserted value, count, id, or status names the read that grounds it. `verify_before_assert()` (provenance-guard) is the assert-from-recollection gate, and `verify_claims()` raises on any claim-versus-record mismatch and emits a `[[vloop:…]]` marker — and it fails closed on a vacuous zero-claim check.

Where the gates live, and when a user runs them.

- **Author-time** (dev-side, run against `bundle_src/` when you are *extending* the bundle): `check_sidecar_contract.py` (kernels contract-clean), then `build_crt_science_bundle.py` (rebuild the JSON from source), then `check_bundle_parity.py` (build, strict-bytes, manifest). Never hand-edit `crt_science_bundle.json`.
- **Ship / install-time** (user-run in a Science session, per CS_INSTALL_STARTER_v2.11): `install_crt_science(overwrite=True)`, then the kernel-gate smoke tests (`model_route_gate` rejects Opus 5; `confirm_before_stop` fails with no receipts; `verify_before_assert` fails on a memory-sourced value; `require_receipt` fails when receiptless), then the sidecar acceptance re-probe.
- **Runtime** (session-time, author-run): the verify_claims, require_receipt, and verify_before_assert kernels above, called *before* the claim ships. There is no Claude-Code-style Stop adversary *hook* on Claude Science; the runtime backstop is these *called* kernels, so the discipline is to actually invoke them.

Gate honesty. Some checks cannot run from a given context — for instance, a live `sidecar_gate` re-probe needs the Science account. Name what ran; never claim the unrunnable green.

Your first research session (walkthrough)

1. Install once, in a Science repl cell (per CS_INSTALL_STARTER_v2.11): `exec(open("install_crt_science.py").read()); install_crt_science(overwrite=True)`, then confirm 52 skills / 18 profiles and no `InstallVerificationError`.

2. Open the project conversation, pick a profile, and use `/plan` for the first nontrivial task.
3. State the task concretely (artifact ids in, artifact names out, "save plots as artifacts").
4. Read the returned plan; approve or refine it.
5. Approve, and it runs cells and writes artifacts; watch the outputs.
6. A long fit goes to the background or to `host.delegate` — collect from the artifact when it is done.
7. If a number looks wrong, ask it to reproduce-before-fixing rather than patch.
8. Use `/handoff-brief` to write a cold-start brief before the window fills.
9. Use `doc-pipeline` or `folio-science` for a PDF of the write-up.
10. Record one canonical memory row per durable outcome, and leave closed state as an inert artifact.

Troubleshooting and FAQ

- "It re-asserted a stale fact." Memory is the poison surface — supersede the canonical row in place and retract the old claim; do not leave two "current" rows.
- "A network call was blocked." That host needs a network grant (for example, `kroki.io` for `folio-science` diagrams) — grant it, or use an offline path.
- "No sound when the run finished." The sandbox has no audio device — `audible-alert` reaches you through the browser (open the emitted HTML artifact).
- "Which method do we actually use for X?" Read `host.lineage` (the record), not a docstring or memory (`provenance-over-description`); for km67, `km67-canonical-methods` first.
- "It forgot earlier context." The window auto-summarized; use `/handoff-brief`, then a fresh session, and rely on artifacts and memory to reload.
- "An intermediate vanished on restart." It was written to `/tmp` — save cell intermediates as artifacts or to `./handoff/` (`provenance-guard`).
- "Wrong model, or too slow." Switch the profile's model (full ids only, never Opus 5), or lower effort for trivial work.

Glossary

- **primitive** — one of Claude Science's core capability layers (context, instructions, actions and guardrails, delegation, the durable record, automation); features derive from these.
- **turn loop** — the cycle Claude Science runs each turn: goal → load context and memory → plan → repl and `host.*` calls within permissions → review → repeat.
- **overlay** — the bundle's added profiles, skills, and kernels layered onto base Claude Science, which runs without them.
- **profile** — a system-prompt persona with a curated skill set and tool access, worn as the main persona or dispatched as a delegated child (the analog of a Claude Code subagent).
- **skill** — a packaged capability that auto-loads on a trigger (`host.skills`); some ship a kernel sidecar.
- **kernel / sidecar** — a skill's `kernel.py` of callable gate and helper functions (the analog of a Claude Code hook — call-time, not event-fired).
- **repl / cell** — the persistent Python kernel; work runs as cells in a remote sandbox.

- **artifact** — a durable, versioned object (data, figure, doc, fit) in `host.artifacts`; the unit of the durable record.
- **lineage** — the reproduction code per artifact version (`host.lineage`); the primary record of what produced a result.
- **memory** — durable project or profile notes that auto-recall into context each turn (the persistent layer, and the poison surface).
- **`host.delegate`** — dispatch specialist profiles as child agents (parallel by default); collect via artifacts.
- **model tier (Fable / Opus 4.8 / Sonnet / Haiku)** — capability versus speed and cost; set per child via `resolve_tier` — never Opus 5.
- **effort** — the reasoning budget spent before acting.
- **network grant** — per-host permission for an outbound call from the sandbox.
- **agency dial (`/solo`, `/collab`, `/plan`)** — the autonomy posture: run-to-completion, surface non-trivial calls, or deliberate first.
- **verification kernel** — a called gate (`verify_claims`, `require_receipt`, `verify_before_assert`) that checks a claim against the record before it ships.
- **attempted-untested vs verified-working** — a fix's efficacy is future-dated until measured (`countermeasure-audit`).
- **handoff brief (`/handoff-brief`)** — a pointer doc so the next session loads targeted instead of re-exploring.
- **machine doc vs human doc** — the LLM-optimized `.machine.md` (the authoritative root) versus the human-prose `.md` (the derived twin).
- **atom** — a single preserved fact, rule, or step, unchanged across a machine-to-human translation.

Ready for more?

- **CS_ADVANCED** — the profiles, skills, and kernels architecture; the `host.*` API surface; memory-as-poison-surface; orchestration topologies; and extension authoring on Claude Science.
- **CS_INSTALL_STARTER_v2.11** (at the bundle root) — the paste-ready, user-run install and its acceptance probes.